EACRMS — Ethiopian Athletics Club & Regional Management System

Backend REST API for managing athletes, coaches, clubs, events, payments, and federation policies.

17	70+	30+
MODULES	API ENDPOINTS	DB MODELS

40+
PERMISSIONS

This is a production backend for the Ethiopian Athletics Federation (EAF), providing a comprehensive management platform for athletes, coaches, clubs, events, payments, and federation policies. It integrates with Ethiopia's **Fayda National ID** system for identity verification and **Africa's Talking** for SMS/phone OTP.

Node.js 22	TypeScript	Express 5
RUNTIME	LANGUAGE	HTTP FRAMEWORK

Prisma 6
ORM

CATEGORY	TECHNOLOGY	PURPOSE
Runtime	Node.js 22 (Alpine)	Server runtime

Language	TypeScript 7	Type-safe development
HTTP	Express 5.2	Web framework
ORM	Prisma 6.19	Database access & migrations
Database	PostgreSQL 17	Primary data store
Cache	Redis (ioredis 6)	Caching layer
Validation	Zod 4	Request validation
Auth	jsonwebtoken 9 + bcrypt 6	JWT + password hashing
Email	Postmark / Resend	Email verification & OTP
SMS	Africa's Talking	Phone OTP via SMS
File Upload	Multer 2	File upload handling
Rate Limit	express-rate-limit 8	API rate limiting
API Docs	Swagger (swagger-jsdoc + swagger-ui)	OpenAPI documentation
Container	Docker (multi-stage)	Production deployment
CI/CD	GitHub Actions	Automated build & deploy

17
FEATURE MODULES

6
MIDDLEWARE

7
ERROR CLASSES

20+
ZOD SCHEMAS

```
eacrms-backend/
├── .github/workflows/
│   ├── ci.yml                  # CI: lint, typecheck, test
│   └── deploy.yml              # CD: build → push → SSH deploy
├── prisma/
│   ├── schema.prisma           # Database schema
│   ├── seed.ts                 # Seed: roles, permissions, admin, news
│   └── migrations/             # Auto-generated migrations
├── scripts/
│   ├── printSwaggerPaths.js    # Swagger endpoint listing utility
│   └── printSwaggerPaths.cjs
├── src/
│   ├── app.ts                  # Express app setup (middleware, routes, error handler)
│   ├── server.ts               # HTTP server entry point
│   ├── routes/
│   │   └── index.ts            # Central route aggregator
│   ├── modules/
│   │   ├── auth/               # Authentication (register, login, refresh, logout)
│   │   ├── verification/       # Email & phone OTP verification
│   │   ├── users/              # User management (admin)
│   │   ├── athletes/           # Athlete registration, profiles, CRUD
│   │   ├── clubs/              # Club registration, verification
│   │   ├── coaches/            # Coach registration & management
│   │   ├── events/             # Events, QR check-in system
│   │   ├── fayda/              # Fayda National ID verification
│   │   ├── payments/           # Payment processing (mock TeleBirr)
│   │   ├── policies/           # Federation policy engine
│   │   ├── permissions/        # Permission management
│   │   ├── roles/              # Role management
│   │   ├── authorizations/     # Authorization service (RBAC)
│   │   ├── audit/              # Audit logging
│   │   ├── news/               # News articles
│   │   ├── meta/               # Registration form options
│   │   └── health/             # Health check endpoint
│   ├── middleware/
│   │   ├── auth.middleware.ts           # JWT Bearer authentication
│   │   ├── authorize.middleware.ts      # Permission-based authorization
│   │   ├── validate.middleware.ts       # Zod schema validation
│   │   ├── rateLimit.middleware.ts      # Rate limiting (public/write/upload/search)
│   │   ├── policy-enforcement.middleware.ts  # Federation policy enforcement
│   │   └── asyncHandler.ts             # Async error wrapper
│   ├── errors/
```

```
│   │   ├── AppError.ts         # Base error class
│   │   ├── BadRequestError.ts
│   │   ├── ConflictError.ts
│   │   ├── ForbiddenError.ts
│   │   ├── NotFoundError.ts
│   │   └── UnauthorizedError.ts
│   ├── lib/
│   │   ├── prisma.ts           # Prisma client singleton
│   │   ├── redis.ts            # Redis client + cache helpers
│   │   ├── postmark.ts         # Postmark email client
│   │   └── uploads/            # File upload destination
│   ├── utils/
│   │   ├── jwt.ts              # JWT generation & verification
│   │   ├── phone.ts            # Ethiopian phone normalization
│   │   └── auth-contract.ts    # Auth API contracts
│   ├── constants/
│   │   └── audit-actions.ts    # Audit action constants
│   ├── types/
│   │   └── express.d.ts        # Express type augmentation (req.user)
│   ├── common/                 # Shared utilities
│   └── docs/
│       ├── swagger.ts          # Swagger spec generator
│       ├── athletes.yaml       # OpenAPI spec fragments
│       ├── auth.yaml
│       ├── clubs.yaml
│       ├── coaches.yaml
│       ├── fayda.yaml
│       └── health.yaml
├── tests/
│   └── auth-contract.test.ts   # Auth contract tests
├── Dockerfile                  # Multi-stage Docker build
├── docker-compose.yml          # Local dev compose
├── docker-compose.prod.yml     # Production compose
├── .env.example                # Environment variable template
├── tsconfig.json               # TypeScript config
├── package.json
├── DEPLOYMENT.md               # Deployment guide
└── README.md
```

The project follows a **modular monolith** architecture with clear separation of concerns within each module. Every feature module follows a consistent layered pattern:

Module Layers

LAYER	FILE PATTERN	RESPONSIBILITY
Routes	`*.routes.ts`	HTTP method mapping, middleware composition
Controller	`*.controller.ts`	Request parsing, response formatting, orchestration
Service	`*.service.ts`	Business logic, validation rules, transaction orchestration
Repository	`*.repository.ts`	Prisma queries, data access abstraction
Validation	`*.validation.ts`	Zod schemas for request validation
Types	`*.types.ts` / `dto/`	TypeScript interfaces, DTOs

> **Request flow:** Client → Route → Auth middleware → Authorization middleware → Validate middleware → Policy enforcement → Controller → Service → Repository → Prisma → PostgreSQL

`src/server.ts` — HTTP Server

```
// Loads dotenv, starts Express on 0.0.0.0:PORT (default 5000)
const PORT = parseInt(process.env.PORT ?? "5000", 10);
app.listen(PORT, "0.0.0.0", () => { ... });
```

`src/app.ts` — Express Application

```
// 1. JSON body parser
// 2. Static file serving (/uploads → /uploads directory)
// 3. API routes (/api/v1/*)
// 4. Swagger UI (/api-docs)
// 5. OpenAPI JSON (/api-docs.json)
// 6. Root health JSON (GET /)
// 7. Global error handler (AppError → structured JSON)
```

`src/routes/index.ts` — Central Router

Aggregates all module routers under `/api/v1`:

```
/api/v1/health          → Health check
/api/v1/auth            → Authentication
/api/v1/auth/verify     → Email & phone verification
/api/v1/athletes        → Athletes
/api/v1/athletes/profile → Athlete profile & uploads
/api/v1/clubs           → Clubs
/api/v1/coaches         → Coaches
/api/v1/fayda/*         → Fayda ID verification
/api/v1/users           → User management
/api/v1/events          → Events & check-in
/api/v1/policies        → Federation policies
/api/v1/payments        → Payments
/api/v1/meta/*          → Registration options
/api/v1/news            → News articles
/api/v1/news/upload     → News image uploads
```

Each module (e.g., athletes) consists of these files:

FILE	EXPORTS
`athlete.route.ts`	Express Router with all athlete endpoints
`athlete.controller.ts`	`athleteController` object with handler methods
`athlete.service.ts`	`athleteService` with business logic

`athlete.repository.ts`	`athleteRepository` with Prisma queries
`athlete.validation.ts`	Zod schemas: `createAthleteSchema`, `updateProfileSchema`, etc.
`dto/`	Data Transfer Objects for complex input shapes

PostgreSQL 17 via Prisma ORM. The schema defines **30+ models** covering users, athletes, coaches, clubs, events, payments, policies, news, and more.

ENUM	VALUES	USED BY
`AthleteStatus`	DRAFT, PENDING, FAYDA_VERIFIED, APPROVED, ACTIVE, SUSPENDED, REJECTED	Athlete, Coach
`UserStatus`	ACTIVE, PENDING, SUSPENDED, DELETED	User
`EventStatus`	DRAFT, PENDING_APPROVAL, PUBLISHED, REJECTED, CANCELLED	Event
`ClubVerificationStatus`	PENDING, VERIFIED, REJECTED, SUSPENDED	Club
`Gender`	MALE, FEMALE	Athlete
`BloodType`	A_POSITIVE, A_NEGATIVE, B_POSITIVE, B_NEGATIVE, AB_POSITIVE, AB_NEGATIVE, O_POSITIVE, O_NEGATIVE	Athlete
`DominantHand`	LEFT, RIGHT, AMBIDEXTROUS	Athlete
`DominantFoot`	LEFT, RIGHT, BOTH	Athlete

`RegistrationSource`	SELF, CLUB_ADMIN	Athlete, Coach
`PolicyScope`	CLUB, EVENT, ATHLETE_PARTICIPATION	Policy
`PolicyStatus`	ACTIVE, INACTIVE	Policy
`PaymentProvider`	MOCK_TELEBIRR	Payment
`PaymentType`	EVENT_REGISTRATION, EVENT_FEE, CLUB_FEE, OTHER	Payment
`PaymentStatus`	PENDING, PAID, FAILED, EXPIRED, REFUNDED	Payment
`EventRegistrationStatus`	PENDING_PAYMENT, CONFIRMED, CANCELLED	EventRegistration
`VerificationType`	EMAIL, PHONE	UserVerification
`VerificationStatus`	PENDING, VERIFIED, EXPIRED	UserVerification
`FaydaVerificationStatus`	PENDING, OTP_SENT, CONFIRMED, FAILED, EXPIRED	FaydaVerification
`NewsCategory`	CHAMPIONSHIP, TRAINING, ANNOUNCEMENT, COMMUNITY, RECOGNITION, GENERAL	News
`PersonalBestScope`	ALL_TIME, SEASON	PersonalBest
`EventAttendeeType`	ATHLETE, CLUB	EventAttendee
`PolicyAuditAction`	CREATED, UPDATED, ASSIGNED, UNASSIGNED	PolicyAuditLog

User

FIELD	TYPE	NOTES

id	String (cuid)	Primary key
email	String? (unique)	Optional email
password	String	Bcrypt hashed
firstName / lastName	String	
phoneNumber	String? (unique)	E.164 format
status	UserStatus	Default: PENDING
emailVerified / phoneVerified	Boolean	Default: false

Relations: roles, athlete, coaches, verifiedClubs, verifications, eventsCreated, eventsApproved, policiesCreated, payments, eventRegistrations

Athlete

FIELD	TYPE	NOTES
id	String (cuid)	Primary key
userId	String (unique)	FK → User
sportId	String?	FK → Sport
clubId / clubName	String?	FK → Club
dateOfBirth	DateTime	
gender	Gender	MALE / FEMALE
nationality	String	
status	AthleteStatus	DRAFT → PENDING → FAYDA_VERIFIED → APPROVED → ACTIVE
faydaVerified	Boolean	

faydaNin	String? (unique)	Fayda National ID Number
height / weight	Int?	cm / kg
dominantHand / dominantFoot	Enum?	
bloodType	BloodType?	
position / jerseyNumber	String? / Int?	
registrationSource	RegistrationSource	SELF / CLUB_ADMIN

Relations: user, sport, club, faydaVerifications, eventAttendees, eventRegistrations, payments, athleteSports, personalBests, trainingLogs, weightLogs

Club

FIELD	TYPE	NOTES
id	String (cuid)	
name	String	
shortName / email / phone	String?	
address / city / region	String?	
registrationNumber / licenseNumber	String? (unique)	
verificationStatus	ClubVerificationStatus	PENDING → VERIFIED / REJECTED / SUSPENDED
verifiedBy / verifiedAt	String? / DateTime?	FK → User

Coach

Similar to Athlete — linked to a User, optional Sport and Club, with license number, specialization, years of experience, and Fayda verification.

Event

FIELD	TYPE	NOTES
id	String (cuid)	
title / description	String / String?	
category	String	
rules	String	
schedule	Json	
status	EventStatus	DRAFT → PENDING_APPROVAL → PUBLISHED
disciplines	Json?	
bannerUrl	String?	
registrationDeadline	DateTime?	
createdById / approvedById	String / String?	FK → User

Relations: statusHistory, policyAssignments, attendees (QR check-in), registrations, payments

Payment

FIELD	TYPE	NOTES

reference	String (unique)	Payment reference ID
provider	PaymentProvider	MOCK_TELEBIRR
type	PaymentType	EVENT_REGISTRATION, etc.
status	PaymentStatus	PENDING → PAID / FAILED / EXPIRED / REFUNDED
amount	Decimal(12,2)	
currency	String	Default: ETB
externalTransactionId	String? (unique)	
providerPayload	Json?	

Policy

Federation-wide policies with configurable rules (JSON). Scopes: CLUB, EVENT, ATHLETE_PARTICIPATION. Policies can block actions or require extra permissions. Full audit trail via PolicyAuditLog.

News

News articles with title, short description, content, category (CHAMPIONSHIP, TRAINING, ANNOUNCEMENT, COMMUNITY, RECOGNITION, GENERAL), featured flag, and image support.

```
User ──1:1── Athlete        (userId unique)
User ──1:1── Coach          (userId unique)
```

```
User ──M:N── Role            (via UserRole)
Role ──M:N── Permission      (via RolePermission)
User ──1:N── AuditLog
User ──1:N── Verification    (email/phone)
User ──1:N── Event           (createdBy / approvedBy)
User ──1:N── Policy          (createdBy / updatedBy)
User ──1:N── Payment
User ──1:N── EventRegistration

Athlete ──M:N── Sport        (via AthleteSport)
Athlete ──M:1── Club
Athlete ──1:N── PersonalBest
Athlete ──1:N── TrainingLog
Athlete ──1:N── WeightLog
Athlete ──1:N── FaydaVerification
Athlete ──M:N── Event        (via EventAttendee)

Club ──1:N── Athlete
Club ──1:N── Coach
Club ──M:N── Policy          (via PolicyAssignment)
Club ──M:N── Event           (via EventAttendee)

Event ──1:N── EventAttendee   (QR tokens + check-in)
Event ──1:N── EventRegistration
Event ──1:N── EventStatusHistory
Event ──M:N── Policy          (via PolicyAssignment)
Event ──1:N── Payment

Payment ──1:N── PaymentStatusHistory
```

`prisma/seed.ts` runs automatically on first deploy (via Dockerfile CMD):

1. **10 Roles:** SUPER_ADMIN, FEDERATION_ADMIN, REGIONAL_ADMIN, CLUB_ADMIN, COACH, ATHLETE, REFEREE, MEDICAL_STAFF, FINANCE_OFFICER, EVENT_MANAGER
2. **40+ Permissions:** user:*, role:*, club:*, athlete:*, coach:*, competition:*, event:*, policy:*, payment:*, report:*, audit:*, fayda:*, news:*
3. **Role-Permission mappings:** SUPER_ADMIN gets all; others get scoped permissions
4. **Default Super Admin:** admin@eacrms.local / ChangeMe123!
5. **6 Seed News Articles:** Sample content for the news module

Base path: `/api/v1/auth`

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/register`	PUBLIC	Register a new user account
POST	`/login`	PUBLIC	Login with email + password, returns access & refresh tokens
POST	`/refresh`	PUBLIC	Refresh access token using refresh token
POST	`/logout`	PUBLIC	Invalidate refresh token
GET	`/me`	AUTH	Get current user profile

Base path: `/api/v1/auth/verify`

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/email`	PUBLIC	Verify email with OTP code
POST	`/phone/public`	PUBLIC	Verify phone by number (post-registration)
POST	`/phone/request`	AUTH	Request phone OTP (sends via Africa's Talking)
POST	`/phone`	AUTH	Verify phone with OTP code
POST	`/resend`	AUTH	Resend verification (email or phone)
GET	`/status`	AUTH	Check verification status for current user

Base path: `/api/v1/users` — Admin-only endpoints

METHOD	ENDPOINT	PERMISSION	DESCRIPTION
GET	`/`	`user:view`	List all users
PATCH	`/:id/activate`	`user:update`	Activate a user
PATCH	`/:id/deactivate`	`user:update`	Deactivate a user
DELETE	`/:id`	`user:delete`	Delete a user

Base path: `/api/v1/athletes`

Public Routes

METHOD	ENDPOINT	DESCRIPTION
GET	`/public`	Public fan-facing athlete list
GET	`/public/:id`	Public athlete detail by ID
POST	`/register`	Self-registration (athlete signup)

Authenticated Routes

METHOD	ENDPOINT	PERMISSION	DESCRIPTION
POST	`/register/by-admin`	`athlete:create`	Club admin registers athlete
GET	`/profile`	—	Get own athlete profile
PATCH	`/profile`	—	Update own profile

GET	`/profile/fayda`	—	Get Fayda verification for profile
GET	`/profile/personal-bests`	—	List personal bests
POST	`/profile/personal-bests`	—	Add personal best
GET	`/profile/training-logs`	—	List training logs
POST	`/profile/training-logs`	—	Add training log
GET	`/profile/weight-logs`	—	List weight logs
POST	`/profile/weight-logs`	—	Add weight log
GET	`/applications`	—	View own applications

Admin Routes

METHOD	ENDPOINT	PERMISSION	DESCRIPTION
GET	`/`	`athlete:view`	List all athletes
GET	`/search`	`athlete:view`	Search athletes
GET	`/status/:status`	`athlete:view`	Filter by status
PATCH	`/:id/approve`	`athlete:update`	Approve athlete
PATCH	`/:id/reject`	`athlete:update`	Reject athlete
PATCH	`/:id/activate`	`athlete:update`	Activate athlete
PATCH	`/:id/suspend`	`athlete:update`	Suspend athlete
GET	`/:id`	`athlete:view`	Get athlete by ID
DELETE	`/:id`	`athlete:delete`	Delete athlete

Base path: `/api/v1/clubs`

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	`/verified`	PUBLIC	List verified clubs
GET	`/public/:id`	PUBLIC	Public club detail
POST	`/register`	PUBLIC	Register a new club
GET	`/`	AUTH	List all clubs
GET	`/pending`	`club:approve`	List pending clubs
GET	`/:id`	AUTH	Get club by ID
PATCH	`/:id/approve`	`club:approve`	Approve club
PATCH	`/:id/reject`	`club:approve`	Reject club
PATCH	`/:id/suspend`	`club:approve`	Suspend club
DELETE	`/:id`	`club:delete`	Delete club

Base path: `/api/v1/coaches`

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/register`	PUBLIC	Self-registration
POST	`/register/by-admin`	`coach:create`	Club admin registers coach
GET	`/profile`	AUTH	Get own profile
GET	`/`	`coach:view`	List all coaches

GET	`/status/:status`	`coach:view`	Filter by status
GET	`/club/:clubId`	`coach:view`	List coaches by club
PATCH	`/:id/approve`	`coach:update`	Approve coach
PATCH	`/:id/reject`	`coach:update`	Reject coach
PATCH	`/:id/activate`	`coach:update`	Activate coach
PATCH	`/:id/suspend`	`coach:update`	Suspend coach
GET	`/:id`	`coach:view`	Get coach by ID
DELETE	`/:id`	`coach:delete`	Delete coach

Base path: `/api/v1/events`

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	`/published`	PUBLIC	List published events
GET	`/:id/detail`	PUBLIC	Event detail (public)
GET	`/`	`event:view`	List all events (admin)
POST	`/`	`event:create`	Create event
GET	`/:id`	`event:view`	Get event by ID
PATCH	`/:id/submit`	`event:create`	Submit for approval (policy-enforced)
PATCH	`/:id/approve`	`event:approve`	Approve event
PATCH	`/:id/reject`	`event:approve`	Reject event
PATCH	`/:id/status`	`event:override`	Override event status (super admin)

Event Check-in (QR)

METHOD	ENDPOINT	PERMISSION	DESCRIPTION
POST	`/:eventId/qr-tokens`	`event:checkin`	Generate QR token for attendee
POST	`/:eventId/check-ins/scan`	`event:checkin`	Scan QR token for check-in
GET	`/:eventId/check-ins`	`event:view`	List check-ins for event

Event Registration & Payments

METHOD	ENDPOINT	PERMISSION	DESCRIPTION
POST	`/:eventId/registrations`	`event:register`	Register athlete for event (creates payment)

Base path: `/api/v1`

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/fayda/initiate`	PUBLIC	Initiate stateless verification (pre-registration)
POST	`/fayda/verify/confirm`	PUBLIC	Confirm OTP for stateless flow
GET	`/fayda/verify/:verificationId`	PUBLIC	Get verification status & data
POST	`/athletes/:athleteId/fayda/initiate`	`fayda:verify`	Initiate for existing athlete
GET	`/athletes/:athleteId/fayda/status`	`fayda:verify`	Get athlete's Fayda status

POST	`/coaches/:coachId/fayda/initiate`	`fayda:verify`	Initiate for existing coach
GET	`/coaches/:coachId/fayda/status`	`fayda:verify`	Get coach's Fayda status
POST	`/fayda/verify/:verificationId/confirm`	AUTH	Confirm OTP (athlete/coach flow)

> **Fayda flow:** Initiate (send NIN) → OTP sent → Confirm OTP → Verification status updated. Supports both stateless (pre-registration) and authenticated flows. Currently uses a mock provider; swap `fayda.mock.ts` for production API.

Base path: `/api/v1/payments`

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/mock/webhook`	PUBLIC	Mock payment webhook (TeleBirr simulation)
GET	`/history`	AUTH	Payment history for current user
GET	`/:paymentId/status`	AUTH	Check payment status

Base path: `/api/v1/policies`

METHOD	ENDPOINT	PERMISSION	DESCRIPTION
GET	`/relevant`	AUTH	Policies relevant to current user's clubs/events

GET	`/`	`policy:view`	List all policies
POST	`/`	`policy:create`	Create policy
GET	`/:id`	`policy:view`	Get policy by ID
PATCH	`/:id`	`policy:update`	Update policy
POST	`/:id/assignments`	`policy:update`	Assign policy to club or event
DELETE	`/:id/assignments/:assignmentId`	`policy:update`	Unassign policy
GET	`/:id/audit`	`policy:view`	View audit log for policy

Base path: `/api/v1/news`

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	`/`	PUBLIC	List published news
GET	`/:id`	PUBLIC	Get news article by ID
GET	`/admin`	`news:view`	List all news (admin)
POST	`/admin`	`news:create`	Create news article
PATCH	`/admin/:id`	`news:update`	Update news article
DELETE	`/admin/:id`	`news:delete`	Delete news article

Base path: `/api/v1/meta`

METHOD	ENDPOINT	DESCRIPTION
GET	`/registration-options`	Public — returns disciplines, clubs, and regions for registration forms

---

Base path: `/api/v1/health`

METHOD	ENDPOINT	DESCRIPTION
GET	`/`	Returns `{"status": "OK"}`

Additional health endpoints:

METHOD	ENDPOINT	DESCRIPTION
GET	`/`	Root — returns `{"name":"EACRMS Backend","version":"1.0.0","status":"running"}`
GET	`/api-docs`	Swagger UI
GET	`/api-docs.json`	OpenAPI JSON spec

The system uses a full **Role-Based Access Control** system with 10 roles and 40+ granular permissions. Permissions follow the format `resource:action` (e.g., `athlete:create`).

Role → Permission Matrix

ROLE	KEY PERMISSIONS
SUPER_ADMIN	**All permissions**

FEDERATION_ADMIN	club:*, athlete:create/update, coach:create/update, event:approve/checkin/view, policy:*, payment:approve, audit:view, fayda:verify, news:*
REGIONAL_ADMIN	club:create/update, athlete:create, coach:create, competition:view, report:view
CLUB_ADMIN	athlete:create/update, coach:create/update, competition:view
EVENT_MANAGER	event:create/view/checkin/register
COACH	athlete:view/update, competition:view
ATHLETE	competition:view, event:register
FINANCE_OFFICER	payment:create/approve/view, report:view
MEDICAL_STAFF	athlete:view, fayda:verify
REFEREE	competition:view, report:view

Full Permission List

PERMISSION	DESCRIPTION
`user:create / view / update / delete`	User management
`role:create / update / delete`	Role management
`club:create / update / approve / delete`	Club management
`athlete:create / view / update / delete`	Athlete management
`coach:create / update / view`	Coach management
`competition:create / update / publish / view`	Competition management

`event:create / view / approve / override / checkin / register`	Event lifecycle
`policy:create / update / view`	Federation policies
`payment:create / approve / view`	Payment operations
`report:view`	Reports
`audit:view`	Audit logs
`fayda:verify`	Fayda ID verification
`news:create / view / update / delete`	News management

MIDDLEWARE	FILE	PURPOSE
`authenticate`	auth.middleware.ts	Verifies Bearer JWT token in Authorization header. Decodes payload and attaches `req.user` with `{ userId, email, roles }`. Throws UnauthorizedError on missing/invalid/expired token.
`authorize(permission)`	authorize.middleware.ts	Higher-order middleware. Checks if the authenticated user has the specified permission via the authorization service (RBAC lookup through UserRole → RolePermission). Throws ForbiddenError if not.
`validate(schema)`	validate.middleware.ts	Parses and validates `req.body` against a Zod schema. Replaces body with parsed output. Returns 400 with field-level error details on failure.

`enforcePolicies(scope, resolveTarget)`	policy-enforcement.middleware.ts	Loads active policies for the given scope (CLUB/EVENT) and target. Blocks action if `rules.blocked === true` or if `rules.requiredPermissions` are not met.
`asyncHandler(handler)`	asyncHandler.ts	Wraps async route handlers to forward rejected promises to Express error middleware.

LIMITER	WINDOW	MAX REQUESTS	USAGE
`publicLimiter`	1 minute	100 / IP	Public read endpoints
`writeLimiter`	1 minute	20 / IP	Write operations (POST/PUT/DELETE)
`uploadLimiter`	1 minute	10 / IP	File uploads
`searchLimiter`	1 minute	60 / IP	Search/browse endpoints

All limiters use standard headers (`RateLimit-*`) and return structured JSON on limit exceeded.

Access Token

- **Payload:** `{ userId, email?, roles[] }`
- **Default expiry:** 15 minutes (configurable via `JWT_ACCESS_EXPIRES_IN`)
- **Secret:** `JWT_ACCESS_SECRET` (required)

Refresh Token

- **Payload:** Same as access token

- **Default expiry:** 7 days (configurable via `JWT_REFRESH_EXPIRES_IN`)
- **Secret:** `JWT_REFRESH_SECRET` (required)

Flow: Login → receive both tokens → use access token in `Authorization: Bearer <token>` → refresh when expired → logout invalidates refresh token.

Copy `.env.example` to `.env` and fill in values:

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
`DATABASE_URL`	✅	—	PostgreSQL connection string
`JWT_ACCESS_SECRET`	✅	—	Secret for signing access tokens
`JWT_REFRESH_SECRET`	✅	—	Secret for signing refresh tokens
`JWT_ACCESS_EXPIRES_IN`	❌	15m	Access token expiry
`JWT_REFRESH_EXPIRES_IN`	❌	7d	Refresh token expiry
`PORT`	❌	5000	Server port
`NODE_ENV`	❌	development	development / production
`RESEND_API_KEY`	❌	—	Resend email API key

`RESEND_FROM_EMAIL`	❌	onboarding@resend.dev	Sender email address
`POSTMARK_API_TOKEN`	❌	—	Postmark email API token
`POSTMARK_SENDER`	❌	onboarding@postmarkapp.com	Postmark sender
`AT_USERNAME`	❌	sandbox	Africa's Talking username
`AT_API_KEY`	❌	—	Africa's Talking API key
`AT_SENDER_ID`	❌	EACRMS	SMS sender ID
`REDIS_URL`	❌	redis://localhost:6379	Redis connection string
`EXPOSE_OTP`	❌	—	Set "true" in dev to expose OTP in responses
`MOCK_PAYMENT_WEBHOOK_SECRET`	❌	—	Secret for mock payment webhook

Multi-stage Dockerfile

The Dockerfile uses a two-stage build for optimized production images:

1. **Builder stage:** Installs all dependencies, generates Prisma client, compiles TypeScript
2. **Production stage:** Installs only production dependencies, copies compiled output + Prisma client

```
FROM node:22-alpine AS builder
# npm ci → prisma generate → tsc

FROM node:22-alpine AS production
# npm ci --omit=dev → copy dist + prisma client
CMD ["sh", "-c", "npx prisma migrate deploy && npx prisma db seed && node dist/ser
```

Docker Compose (Local Dev)

```
# docker-compose.yml
services:
  postgres:    # PostgreSQL 17 Alpine, port 5432, persistent volume
  app:         # Builds from Dockerfile, port 5000, depends_on postgres healthy
```

Docker Compose (Production)

```
# docker-compose.prod.yml
services:
  postgres:    # PostgreSQL 17 Alpine, persistent volume, healthcheck
  app:         # Uses pre-built image from GHCR, configurable HOST_PORT
```

CI Workflow (`.github/workflows/ci.yml`)

Triggers: push to main, pull requests, workflow_call

Steps:

1. Checkout code

2. Setup Node.js 22 (with npm cache)
3. Install dependencies (`npm ci`)
4. Generate Prisma client
5. Validate Prisma schema
6. Compile TypeScript

Services: PostgreSQL 17 Alpine (healthchecked)

Deploy Workflow (`.github/workflows/deploy.yml`)

Triggers: push to main, manual (workflow_dispatch)
Jobs:

1. **build-and-push:** Build Docker image → Push to GHCR (tagged with commit SHA + latest)
2. **deploy:** SSH into server → Pull image → Docker Compose up → Cleanup unused images

Required GitHub Secrets:

- `SSH_PRIVATE_KEY` — SSH key for deploy user
- `DEPLOY_HOST` — Server IP
- `DEPLOY_USER` — SSH username
- `DEPLOY_PORT` — SSH port (default: 22)
- `GHCR_TOKEN` / `GHCR_USERNAME` — Registry credentials

Server Layout

```
/var/www/eacrms-backend/
├── docker-compose.prod.yml
```

```
├── .env                        # Production secrets (never in git)
└── backups/                    # Database backups
```

Deploy Commands

```
# Pull and restart
cd /var/www/eacrms-backend
docker pull ghcr.io/<owner>/eacrms-backend:<commit-sha>
docker compose -f docker-compose.prod.yml up -d --remove-orphans

# Rollback to previous image
# Update IMAGE_TAG in .env, then:
docker compose -f docker-compose.prod.yml up -d

# Check status
docker compose -f docker-compose.prod.yml ps
docker logs --follow eacrms-app
```

Database Backup

```
# Backup
docker exec -t eacrms-postgres pg_dump -U postgres eacrms \
  > backups/db_$(date +%F_%T).sql

# Restore
cat backups/db_2026-01-01_12:00:00.sql | \
  docker exec -i eacrms-postgres psql -U postgres -d eacrms
```

Error Class Hierarchy

```
AppError (base, statusCode + message)
├── BadRequestError       (400)
├── UnauthorizedError     (401)
├── ForbiddenError        (403)
```

```
├── NotFoundError         (404)
└── ConflictError         (409)
```

Global Error Handler

All unhandled errors are caught by the Express error middleware in `app.ts`

```
// AppError instances → { success: false, message: err.message }
// Unknown errors → { success: false, message: "Internal server error." }
```

Validation Errors

Zod validation errors return structured field-level details:

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": [
    { "field": "email", "message": "Invalid email" },
    { "field": "password", "message": "String must contain at least 8 characters" }
  ]
}
```

All request bodies are validated with **Zod 4** schemas. The `validate` middleware parses the body and replaces `req.body` with the validated, typed output.

Each module defines its own schemas in `*.validation.ts` files, e.g.:

- `createAthleteSchema` — athlete registration
- `createEventSchema` — event creation
- `createCoachSchema` — coach registration
- `registerClubSchema` — club registration
- `initiateVerificationSchema` — Fayda NIN input
- `confirmOtpSchema` — OTP confirmation
- `createPolicySchema` — policy creation
- `createNewsSchema` / `updateNewsSchema` — news articles

Redis is used as a caching layer via `ioredis` with graceful degradation:

```
// lib/redis.ts — cache helpers

// Get from cache or compute + cache
const data = await cacheGet<T>(key, ttlSeconds, async () => {
  // fallback: query database
});

// Invalidate by pattern
await cacheInvalidate("athletes:*");
```

- Max retries: 3, with exponential backoff (200ms–2000ms)
- **Graceful degradation:** If Redis is unavailable, all operations fall through to the database without errors
- Cache writes that fail are logged but non-critical

Source Files (76 module files + support)

DIRECTORY	FILES	DESCRIPTION
`src/`	app.ts, server.ts	Entry points
`src/routes/`	index.ts	Central route aggregator
`src/modules/auth/`	5 files	Authentication (register, login, refresh, logout, me)
`src/modules/verification/`	8 files	Email & phone OTP verification
`src/modules/users/`	3 files	User management (admin CRUD)

`src/modules/athletes/`	7 files + dto/	Athlete registration, profiles, personal bests, training logs
`src/modules/clubs/`	5 files	Club registration & verification
`src/modules/coaches/`	5 files + dto/	Coach registration & management
`src/modules/events/`	8 files	Events, QR check-in, status management
`src/modules/fayda/`	7 files	Fayda National ID verification (mock + types)
`src/modules/payments/`	4 files	Payment processing (mock TeleBirr)
`src/modules/policies/`	5 files	Federation policy engine & audit
`src/modules/news/`	6 files	News articles & image uploads
`src/modules/authorizations/`	3 files	RBAC authorization service
`src/modules/permissions/`	—	Permission management
`src/modules/roles/`	—	Role management
`src/modules/audit/`	5 files	Audit logging
`src/modules/meta/`	2 files	Registration form options (disciplines, clubs, regions)
`src/modules/health/`	1 file	Health check endpoint
`src/middleware/`	6 files	Auth, authorize, validate, rate-limit, policy, async
`src/errors/`	6 files	Error class hierarchy
`src/lib/`	3 files + uploads/	Prisma, Redis, Postmark clients
`src/utils/`	3 files	JWT, phone normalization, auth contracts
`src/docs/`	7 files	Swagger spec generator + OpenAPI fragments

`src/constants/`	1 file	Audit action constants
`src/types/`	1 file	Express type augmentation

Configuration Files

FILE	PURPOSE
`package.json`	Dependencies, scripts (dev/build/start)
`tsconfig.json`	TypeScript: ES2022, NodeNext module, strict, sourceMap
`Dockerfile`	Multi-stage Docker build (builder + production)
`docker-compose.yml`	Local dev: postgres + app
`docker-compose.prod.yml`	Production: postgres + pre-built image
`.env.example`	Environment variable template
`.gitignore`	Git ignore rules
`.dockerignore`	Docker ignore rules
`DEPLOYMENT.md`	Full deployment guide

EACRMS Backend — v1.0.0

Ethiopian Athletics Club & Regional Management System

Generated: August 26, 2026

Repository: github.com/Ged45/eacrms-backend